Sentiment classifier

🎯 Today's goal

By the end of this hour you will:

- train a model that reads a review and says whether it is **positive** or **negative**
- chain TF-IDF and a classifier into one **pipeline**
- check it with accuracy and a confusion matrix, and read its mistakes
- see which words the model relies on, and compare it with **Naive Bayes**
- understand where it fails: "not bad", sarcasm and tiny datasets

🤔 Why this matters

Sentiment is the feeling behind a text: happy, unhappy, or somewhere in between. A shop with 50 000 reviews can't read them all, but a sentiment classifier can sort them in a second and point out the angry ones. The same idea sorts emails into spam and not spam, or support tickets into topics.

You already have every piece. From Day 18: a classifier learns from labelled examples. From yesterday: TF-IDF turns text into numbers. Today we join them. It's **supervised** learning (Day 16): we give the model reviews together with the right answer (the **label**), and it learns which words go with which label.

🏷️ Our data: 80 labelled reviews

Here are 80 **made-up** short reviews from a made-up online shop. Each one is a pair: `(text, label)`. Half are positive and half negative, so the classes are **balanced**. (Tomorrow's capstone uses the very same list.)

```
# 80 made-up reviews from a made-up online shop (invented for this lesson)
labelled_reviews = [
    ("Great sound and the battery lasts all day", "positive"),
    ("I love these headphones, so comfortable", "positive"),
    ("Fast delivery and excellent quality", "positive"),
    ("Works perfectly, I would buy it again", "positive"),
    ("Excellent value, great price", "positive"),
    ("The kettle is fast and looks lovely", "positive"),
    ("Really happy with this lamp, lovely warm light", "positive"),
    ("Best backpack I have ever owned, love it", "positive"),
    ("Strong and light, great for travel", "positive"),
    ("The blender is powerful and easy to clean", "positive"),
    ("Great phone case, fits perfectly", "positive"),
    ("Comfortable shoes, I wear them every day", "positive"),
    ("Great gift, my sister loved it", "positive"),
    ("Arrived early and works great", "positive"),
    ("Fast charger, highly recommend", "positive"),
    ("Beautiful colour and soft fabric, love it", "positive"),
    ("The keyboard is excellent and quiet", "positive"),
    ("Amazing picture quality, really happy", "positive"),
    ("Easy to set up, the app is simple and good", "positive"),
    ("Good quality for a low price", "positive"),
    ("Not bad at all, does the job well", "positive"),
    ("Really good, better than I expected", "positive"),
    ("Perfect size and well made", "positive"),
    ("The coffee machine is excellent, great taste", "positive"),
    ("Happy with my purchase, nice and sturdy", "positive"),
    ("Lovely mug, keeps my tea hot", "positive"),
    ("The speaker is loud and clear, love it", "positive"),
    ("Five stars, excellent customer service", "positive"),
    ("Cheap but works really well", "positive"),
    ("Exactly as described, very happy", "positive"),
    ("Smooth, quiet and fast, I love it", "positive"),
    ("Great for running, light and comfortable", "positive"),
    ("My kids love this game", "positive"),
    ("Nice design and the battery is great", "positive"),
    ("The best toaster I have had, works perfectly", "positive"),
    ("The watch looks great and works well", "positive"),
    ("Sturdy table, easy to build, recommend", "positive"),
    ("I recommend it to everyone, great buy", "positive"),
    ("The pillow is soft and comfortable", "positive"),
    ("Good sound, good price, happy customer", "positive"),
    ("Terrible quality, it broke after a week", "negative"),
    ("The battery is poor and dies after two hours", "negative"),
    ("Not good, the sound is poor", "negative"),
    ("Arrived late and the box was broken", "negative"),
    ("Waste of money, do not buy", "negative"),
    ("The kettle leaks, terrible", "negative"),
    ("Very disappointed, it stopped working", "negative"),
    ("Cheap plastic, it broke on day one", "negative"),
    ("The app keeps crashing, useless", "negative"),
    ("Too small and uncomfortable, returned it", "negative"),
```

```
    ("Poor quality, the strap broke", "negative"),
    ("The charger is slow and stopped working", "negative"),
    ("Awful customer service, never replied", "negative"),
    ("Not worth the price, disappointed", "negative"),
    ("The colour faded after one wash, poor", "negative"),
    ("Terrible screen, the picture is blurry", "negative"),
    ("Returned it, the worst blender I have used", "negative"),
    ("The shoes are uncomfortable and hurt my feet", "negative"),
    ("Noisy fan and the remote is useless", "negative"),
    ("Broken on arrival, very disappointed", "negative"),
    ("The lamp is awful and the light is cold", "negative"),
    ("Bad smell and the fabric is rough", "negative"),
    ("Hard to set up, the instructions are useless", "negative"),
    ("The speaker is terrible at high volume", "negative"),
    ("I would not recommend this, awful", "negative"),
    ("The mug broke in the dishwasher", "negative"),
    ("Slow delivery and the wrong size", "negative"),
    ("The keyboard is bad and some keys stick", "negative"),
    ("Stopped working after a month, waste of money", "negative"),
    ("Awful taste, the coffee machine leaks", "negative"),
    ("Poor table, it wobbles", "negative"),
    ("The game is boring, a waste", "negative"),
    ("Overpriced and badly made, disappointed", "negative"),
    ("The backpack zip broke, terrible", "negative"),
    ("Disappointed, it looks nothing like the photo", "negative"),
    ("The pillow is too hard and uncomfortable", "negative"),
    ("Not happy, the watch stopped working", "negative"),
    ("Uncomfortable and the sound is poor", "negative"),
    ("The toaster is the worst, returned it", "negative"),
    ("Bad quality, bad price, unhappy customer", "negative"),
]

texts = [text for text, label in labelled_reviews]
labels = [label for text, label in labelled_reviews]

print(len(texts), "reviews")
print(texts[0], "->", labels[0])
print(texts[40], "->", labels[40])
```

```
80 reviews
Great sound and the battery lasts all day -> positive
Terrible quality, it broke after a week -> negative
```

`for text, label in labelled_reviews` unpacks each pair into two names, like the tuple unpacking from Day 6.

A quick look with pandas (Day 10) confirms the balance:

```
import pandas as pd

df = pd.DataFrame(labelled_reviews, columns=["review", "label"])
print(df["label"].value_counts())
df.sample(5, random_state=42)
```

```
label
positive    40
negative    40
Name: count, dtype: int64
```

	review	label
30	Smooth, quiet and fast, I love it	positive
0	Great sound and the battery lasts all day	positive
22	Perfect size and well made	positive
31	Great for running, light and comfortable	positive
18	Easy to set up, the app is simple and good	positive

✂️ Train and test

Just like on Day 16, we hide some reviews from the model so we can test it on text it has never seen. `stratify=labels` keeps the 50/50 balance in both parts:

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    texts, labels, test_size=0.25, stratify=labels, random_state=42
)

print(len(X_train), "reviews to learn from,", len(X_test), "kept back for testing")
print(y_test.count("positive"), "positive and", y_test.count("negative"), "negative in
the test set")
```

```
60 reviews to learn from, 20 kept back for testing
10 positive and 10 negative in the test set
```

🔗 One pipeline: TF-IDF → logistic regression

On Day 18 you used `make_pipeline` to join a scaler and a model. Here we join a `TfidfVectorizer` and a `LogisticRegression`. The pipeline takes raw text: `fit` learns the

vocabulary *and* trains the classifier, and `predict` turns new text into numbers with the same vocabulary before classifying it. No chance of forgetting a step.

First, a mistake many people make: asking for predictions before training.

💡 This example has a mistake on purpose. Run it and read the error.

```
from sklearn.pipeline import make_pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression

model = make_pipeline(TfidfVectorizer(), LogisticRegression())
model.predict(["I love it"])
```

⚠️ **NotFittedError**

```
Traceback (most recent call last):
  line 6, in <module>
    model.predict(["I love it"])
    ~~~~~~~~~~~~~~^^^^^^^^^^^^^^^
sklearn.exceptions.NotFittedError: This Pipeline instance is not fitted yet. Call
'fit' with appropriate arguments before using this estimator.
```

`NotFittedError`: the model hasn't learned anything yet. Call `fit` first, with the training reviews and their labels:

```
model.fit(X_train, y_train)

accuracy = model.score(X_test, y_test)
print(f"Accuracy on {len(X_test)} unseen reviews: {accuracy:.0%}")
```

```
Accuracy on 20 unseen reviews: 95%
```

`:.0%` in an f-string shows a share like 0.95 as a percentage with 0 decimals: 95%.

It got 19 of the 20 hidden reviews right. Now the fun part: reviews nobody has written before. `predict_proba` (Day 18) shows how sure the model is. The columns follow `model.classes_`, which are in A–Z order:

```
new_reviews = [
    "The battery is great and delivery was fast",
    "It broke after two days, awful",
    "Nice colour but the sound is poor",
]

print(model.predict(new_reviews))
print(model.classes_)

chances = pd.DataFrame(model.predict_proba(new_reviews), columns=model.classes_,
index=new_reviews)
chances.round(2)
```

```
['positive' 'negative' 'negative']
['negative' 'positive']
```

	negative	positive
The battery is great and delivery was fast	0.39	0.61
It broke after two days, awful	0.65	0.35
Nice colour but the sound is poor	0.55	0.45

All three are right, but the model is not very *sure* (around 0.6). With only 60 training reviews, each word has been seen just a few times. The mixed third review, half nice and half poor, is closest to 50/50, which is honest.

🔍 Where does it go wrong?

A **confusion matrix** (Day 20) shows which kind of mistakes the model makes. Even better for text: we can simply read the reviews it got wrong.

```
from sklearn.metrics import confusion_matrix

predictions = model.predict(X_test)
cm = confusion_matrix(y_test, predictions, labels=["negative", "positive"])
print(pd.DataFrame(cm, index=["really negative", "really positive"],
                   columns=["said negative", "said positive"]))
print()

for text, true, said in zip(X_test, y_test, predictions):
    if true != said:
        print(f"MISTAKE: {text!r} is {true}, the model said {said}")
```

```
                 said negative  said positive
really negative             10              0
really positive              1              9

MISTAKE: 'Not bad at all, does the job well' is positive, the model said
negative
```

Its only mistake is a positive review that starts with *"Not bad"*. Keep that in mind: we'll come back to it in a moment.

🧠 Which words does the model rely on?

Logistic regression learns one **weight** (number) per vocabulary word, stored in `coef_`. A word with a big positive weight pushes a review towards `"positive"` (the second class in `classes_`); a big negative weight pushes it towards `"negative"`. `named_steps` lets us take the two parts out of the pipeline:

```
vectorizer = model.named_steps["tfidfvectorizer"]
classifier = model.named_steps["logisticregression"]

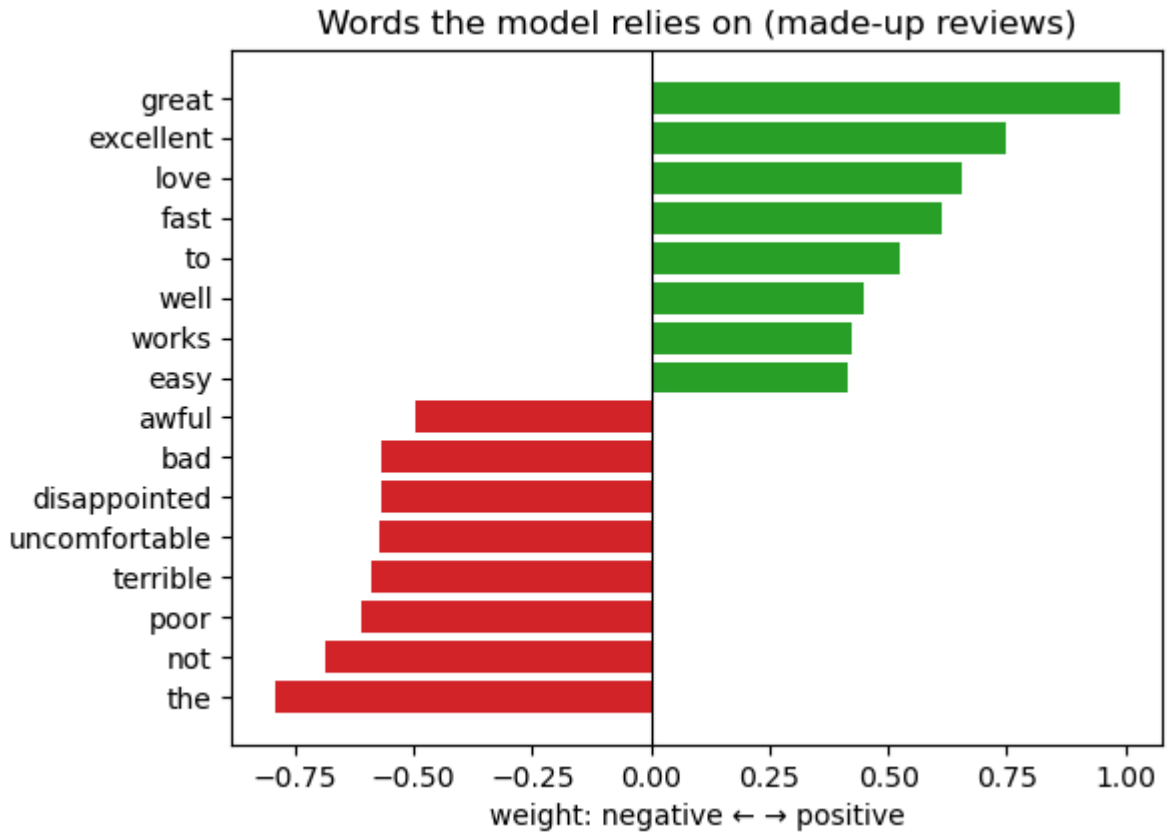
weights = pd.Series(classifier.coef_[0],
index=vectorizer.get_feature_names_out()).sort_values()
print("Most negative words:", list(weights.index[:6]))
print("Most positive words:", list(weights.index[-6:]))
```

```
Most negative words: ['the', 'not', 'poor', 'terrible', 'uncomfortable',
'disappointed']
Most positive words: ['well', 'to', 'fast', 'love', 'excellent', 'great']
```

```
import matplotlib.pyplot as plt

top = pd.concat([weights.head(8), weights.tail(8)])
colours = ["tab:red" if w < 0 else "tab:green" for w in top]

plt.figure(figsize=(6, 4.5))
plt.barh(top.index, top.values, color=colours)
plt.axvline(0, color="black", linewidth=0.8)
plt.title("Words the model relies on (made-up reviews)")
plt.xlabel("weight: negative ← → positive")
plt.show()
```



Most of it makes sense: *great, excellent, love* on one side; *poor, terrible, awful* on the other. But look: `the` and `not` count as negative words! In our small dataset, unhappy customers simply happened to write "the" and "not" more often. The model doesn't understand English: it finds words that go together with a label. Reading the weights is a great way to catch such silly lessons.

🎲 A second opinion: Naive Bayes

Naive Bayes is a classic text classifier. It looks at how often each word appears in positive and in negative training reviews, and combines those chances for a new review. It's called "naive" because it treats every word as if it had nothing to do with its neighbours. It is very fast and works

surprisingly well on text. scikit-learn's version for word counts is `MultinomialNB`; swapping it in is one line:

```
from sklearn.naive_bayes import MultinomialNB

nb_model = make_pipeline(TfidfVectorizer(), MultinomialNB())
nb_model.fit(X_train, y_train)

print(f"Logistic regression: {model.score(X_test, y_test):.0%}")
print(f"Naive Bayes:         {nb_model.score(X_test, y_test):.0%}")
```

```
Logistic regression: 95%
Naive Bayes:         90%
```

Close. With only 20 test reviews, one review is worth 5 percentage points, so don't read too much into a small difference.

🙃 The "not bad" problem

A bag of words has no idea about word order. *"not bad"* is just *not* (a negative word, as we saw) plus *bad* (also negative):

```
tricky = ["not good", "not bad", "not happy", "the battery is not great"]

for text, label in zip(tricky, model.predict(tricky)):
    print(f"{text:26} -> {label}")

print()
print("weight of 'not':", round(weights["not"], 2))
```

```
not good                   -> negative
not bad                    -> negative
not happy                  -> negative
the battery is not great   -> negative

weight of 'not': -0.69
```

`{text:26}` pads the text to 26 characters so the arrows line up.

"not bad" comes out negative: wrong. Yesterday's idea, **bigrams** (`ngram_range=(1, 2)`), lets the model see *"not bad"* as one feature. Let's measure whether it helps here:

```
bigram_model = make_pipeline(TfidfVectorizer(ngram_range=(1, 2)),
LogisticRegression())
bigram_model.fit(X_train, y_train)

print(f"Words only:     {model.score(X_test, y_test):.0%}")
print(f"Words + pairs:  {bigram_model.score(X_test, y_test):.0%}")

pd.DataFrame({"words only": model.predict(tricky),
              "words + pairs": bigram_model.predict(tricky)}, index=tricky)
```

```
Words only:     95%
Words + pairs:  85%
```

	words only	words + pairs
not good	negative	negative
not bad	negative	negative
not happy	negative	negative
the battery is not great	negative	positive

Surprise: on our data, bigrams make things **worse**. To learn that "not bad" is positive, the model must see "not bad" in positive training reviews, and our 60 training reviews contain it zero times. Meanwhile the extra word-pair columns, most of them seen only once, add noise. With thousands of reviews, bigrams usually do help. The lesson is the one from Day 20: **don't guess, measure.**

🚧 Limits

Our classifier is useful, but know what it can't do:

- **Sarcasm:** "Great, it broke after one day. Just great." contains *great* twice.
- **Tiny data:** 80 made-up reviews can't cover the words real customers use. Real systems train on many thousands.
- **New topics:** a model trained on product reviews may fail on film or restaurant reviews, where words like "cheap" or "long" mean something else.
- **Mixed feelings:** "lovely design, terrible battery" doesn't fit into two boxes.

```
sarcastic = "Great, it broke after one day. Just great."
print(model.predict([sarcastic]), model.predict_proba([sarcastic]).round(2))
```

```
['positive'] [[0.39 0.61]]
```

Modern language models (like the ones behind chatbots) read whole sentences in order and handle many of these cases much better. Tomorrow's "What next" section points the way.

⚠️ Common mistakes

- Calling `predict` before `fit` → `NotFittedError`.
- Passing one string instead of a list: use `model.predict(["some text"])`.
- Testing on the training reviews. The score looks great but means nothing; always keep a test set.
- Reading the `predict_proba` columns in the wrong order: check `model.classes_`.
- Believing a small difference in accuracy on a small test set. One review can change it by 5%.

✅ Recap

- **Sentiment analysis** = text classification with labels like positive / negative.
- `make_pipeline(TfidfVectorizer(), LogisticRegression())` goes from raw text to a label.
- Split with `stratify`, `fit` on training text, `score` on test text.
- `coef_` + `get_feature_names_out()` show which words push towards each label.
- `MultinomialNB` (Naive Bayes) is a fast classic alternative.
- Bags of words miss order ("not bad") and sarcasm; bigrams can help, but only with enough data. Measure!

🏋️ Exercises

Write your answer in the space under each exercise. Try it on the computer too, by opening this day's `index.html`. Solutions are in the answer key at the end.

The exercises use a smaller set of 24 made-up reviews, so the boxes stay short. `["positive"] * 12` makes a list with 12 copies of `"positive"`.

1. Split the reviews

Split `texts` and `labels` into `X_train`, `X_test`, `y_train` and `y_test` with `test_size=0.25`, `stratify=labels` and `random_state=42`.

```
from sklearn.model_selection import train_test_split

positive = ["great sound, love it", "excellent quality and fast delivery",
            "works perfectly, great price", "really happy, highly recommend",
            "comfortable and well made", "love the colour, great buy",
            "fast and easy to use", "excellent, would buy again",
            "good value, happy customer", "great battery and lovely design",
            "perfect fit, very comfortable", "easy to set up, works great"]
negative = ["terrible, it broke after a week", "poor quality, waste of money",
            "awful sound, very disappointed", "stopped working, returned it",
            "slow delivery and a broken box", "uncomfortable and too small",
            "bad battery, dies quickly", "useless app, keeps crashing",
            "poor fit, disappointed", "terrible customer service",
            "broke on day one, awful", "waste of time, bad quality"]
texts = positive + negative
labels = ["positive"] * len(positive) + ["negative"] * len(negative)

# Split into X_train, X_test, y_train, y_test
```

Hints:

1. The function returns four things, in this order: `X_train, X_test, y_train, y_test = train_test_split(...)`
2. `train_test_split(texts, labels, test_size=0.25, stratify=labels, random_state=42)`

2. Train a pipeline

Make a pipeline called `model` from a `TfidfVectorizer()` and a `LogisticRegression()`, train it on the training reviews, and store its accuracy on the test reviews in `accuracy`.

```
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression

positive = ["great sound, love it", "excellent quality and fast delivery",
            "works perfectly, great price", "really happy, highly recommend",
            "comfortable and well made", "love the colour, great buy",
            "fast and easy to use", "excellent, would buy again",
            "good value, happy customer", "great battery and lovely design",
            "perfect fit, very comfortable", "easy to set up, works great"]
negative = ["terrible, it broke after a week", "poor quality, waste of money",
            "awful sound, very disappointed", "stopped working, returned it",
            "slow delivery and a broken box", "uncomfortable and too small",
            "bad battery, dies quickly", "useless app, keeps crashing",
            "poor fit, disappointed", "terrible customer service",
            "broke on day one, awful", "waste of time, bad quality"]
texts = positive + negative
labels = ["positive"] * len(positive) + ["negative"] * len(negative)
X_train, X_test, y_train, y_test = train_test_split(
    texts, labels, test_size=0.25, stratify=labels, random_state=42
)

# Make model, fit it, and store the test accuracy in accuracy
```

Hints:

1. `model = make_pipeline(TfidfVectorizer(), LogisticRegression())`
2. Train with `model.fit(X_train, y_train)`.
3. `accuracy = model.score(X_test, y_test)`

3. Predict new reviews

The model below is already trained on all 24 reviews. Store its predicted labels for the three new reviews in `predictions`, and the chance that the **first** new review is **positive** in `positive_chance` (a number between 0 and 1).

```
from sklearn.pipeline import make_pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression

positive = ["great sound, love it", "excellent quality and fast delivery",
            "works perfectly, great price", "really happy, highly recommend",
            "comfortable and well made", "love the colour, great buy",
            "fast and easy to use", "excellent, would buy again",
            "good value, happy customer", "great battery and lovely design",
            "perfect fit, very comfortable", "easy to set up, works great"]
negative = ["terrible, it broke after a week", "poor quality, waste of money",
            "awful sound, very disappointed", "stopped working, returned it",
            "slow delivery and a broken box", "uncomfortable and too small",
            "bad battery, dies quickly", "useless app, keeps crashing",
            "poor fit, disappointed", "terrible customer service",
            "broke on day one, awful", "waste of time, bad quality"]
texts = positive + negative
labels = ["positive"] * len(positive) + ["negative"] * len(negative)
model = make_pipeline(TfidfVectorizer(), LogisticRegression())
model.fit(texts, labels)

new_reviews = ["the sound is great", "it broke, terrible", "fast delivery but
poor quality"]

# Store predictions and positive_chance
```

Hints:

1. `predictions = model.predict(new_reviews)`
2. `model.predict_proba(new_reviews)` gives one row per review and one column per class. `print(model.classes_)` shows the column order.
3. "positive" is the second column: `positive_chance = model.predict_proba(new_reviews)[0, 1]`

4. The most telling words

Using the trained pipeline, find the word with the **biggest** weight (the most positive word) and the word with the **smallest** weight (the most negative word). Store them in `top_positive` and `top_negative`.

```
import pandas as pd
from sklearn.pipeline import make_pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression

positive = ["great sound, love it", "excellent quality and fast delivery",
            "works perfectly, great price", "really happy, highly recommend",
            "comfortable and well made", "love the colour, great buy",
            "fast and easy to use", "excellent, would buy again",
            "good value, happy customer", "great battery and lovely design",
            "perfect fit, very comfortable", "easy to set up, works great"]
negative = ["terrible, it broke after a week", "poor quality, waste of money",
            "awful sound, very disappointed", "stopped working, returned it",
            "slow delivery and a broken box", "uncomfortable and too small",
            "bad battery, dies quickly", "useless app, keeps crashing",
            "poor fit, disappointed", "terrible customer service",
            "broke on day one, awful", "waste of time, bad quality"]
texts = positive + negative
labels = ["positive"] * len(positive) + ["negative"] * len(negative)
model = make_pipeline(TfidfVectorizer(), LogisticRegression())
model.fit(texts, labels)

# Store top_positive and top_negative
```

Hints:

1. Get the parts: `model.named_steps["tfidfvectorizer"]` and `model.named_steps["logisticregression"]`.
2. `weights = pd.Series(classifier.coef_[0], index=vectorizer.get_feature_names_out())`
3. `top_positive = weights.idxmax()` and `top_negative = weights.idxmin()`

5. Challenge: do bigrams help?

Train two pipelines on the same split and compare them on the test reviews:

- `acc_uni`: the accuracy with `TfidfVectorizer()` (single words),
- `acc_bi`: the accuracy with `TfidfVectorizer(ngram_range=(1, 2))`,
- `better`: the text `"unigrams"`, `"bigrams"` or `"same"`, decided by an `if` from the two accuracies.

Your code should still give the right `better` if the data changes.

```
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression

positive = ["great sound, love it", "excellent quality and fast delivery",
            "works perfectly, great price", "really happy, highly recommend",
            "comfortable and well made", "love the colour, great buy",
            "fast and easy to use", "excellent, would buy again",
            "good value, happy customer", "great battery and lovely design",
            "perfect fit, very comfortable", "easy to set up, works great"]
negative = ["terrible, it broke after a week", "poor quality, waste of money",
            "awful sound, very disappointed", "stopped working, returned it",
            "slow delivery and a broken box", "uncomfortable and too small",
            "bad battery, dies quickly", "useless app, keeps crashing",
            "poor fit, disappointed", "terrible customer service",
            "broke on day one, awful", "waste of time, bad quality"]
texts = positive + negative
labels = ["positive"] * len(positive) + ["negative"] * len(negative)
X_train, X_test, y_train, y_test = train_test_split(
    texts, labels, test_size=0.25, stratify=labels, random_state=42
)

# Store acc_uni, acc_bi and better
```

Hints:

1. Build, fit and score the first: `uni = make_pipeline(TfidfVectorizer(), LogisticRegression())`, then `uni.fit(X_train, y_train)` and `acc_uni = uni.score(X_test, y_test)`.
2. Do the same with `TfidfVectorizer(ngram_range=(1, 2))` for `acc_bi`.
3. `if acc_bi > acc_uni: better = "bigrams"`, `elif acc_uni > acc_bi: better = "unigrams"`, `else: better = "same"`

➡️ Tomorrow

The capstone: a review analyzer. On the last day of PyPath you'll combine text cleaning, TF-IDF and today's classifier into one `analyze()` function that reads a batch of new reviews and reports the share of happy customers, the top keywords and a chart. Then we'll look back at everything you've learned in 30 days, and at where to go next.

🔑 Answer key

Try the exercises yourself first! There is often more than one correct answer.

1. Split the reviews

```
from sklearn.model_selection import train_test_split

positive = ["great sound, love it", "excellent quality and fast delivery",
            "works perfectly, great price", "really happy, highly recommend",
            "comfortable and well made", "love the colour, great buy",
            "fast and easy to use", "excellent, would buy again",
            "good value, happy customer", "great battery and lovely design",
            "perfect fit, very comfortable", "easy to set up, works great"]
negative = ["terrible, it broke after a week", "poor quality, waste of money",
            "awful sound, very disappointed", "stopped working, returned it",
            "slow delivery and a broken box", "uncomfortable and too small",
            "bad battery, dies quickly", "useless app, keeps crashing",
            "poor fit, disappointed", "terrible customer service",
            "broke on day one, awful", "waste of time, bad quality"]
texts = positive + negative
labels = ["positive"] * len(positive) + ["negative"] * len(negative)

X_train, X_test, y_train, y_test = train_test_split(
    texts, labels, test_size=0.25, stratify=labels, random_state=42
)
print(len(X_train), len(X_test))
```

2. Train a pipeline

```
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression

positive = ["great sound, love it", "excellent quality and fast delivery",
            "works perfectly, great price", "really happy, highly recommend",
            "comfortable and well made", "love the colour, great buy",
            "fast and easy to use", "excellent, would buy again",
            "good value, happy customer", "great battery and lovely design",
            "perfect fit, very comfortable", "easy to set up, works great"]
negative = ["terrible, it broke after a week", "poor quality, waste of money",
            "awful sound, very disappointed", "stopped working, returned it",
            "slow delivery and a broken box", "uncomfortable and too small",
            "bad battery, dies quickly", "useless app, keeps crashing",
            "poor fit, disappointed", "terrible customer service",
            "broke on day one, awful", "waste of time, bad quality"]
texts = positive + negative
labels = ["positive"] * len(positive) + ["negative"] * len(negative)
X_train, X_test, y_train, y_test = train_test_split(
    texts, labels, test_size=0.25, stratify=labels, random_state=42
)

model = make_pipeline(TfidfVectorizer(), LogisticRegression())
model.fit(X_train, y_train)
accuracy = model.score(X_test, y_test)
print(f"Accuracy: {accuracy:.0%}")
```

3. Predict new reviews

```
from sklearn.pipeline import make_pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression

positive = ["great sound, love it", "excellent quality and fast delivery",
            "works perfectly, great price", "really happy, highly recommend",
            "comfortable and well made", "love the colour, great buy",
            "fast and easy to use", "excellent, would buy again",
            "good value, happy customer", "great battery and lovely design",
            "perfect fit, very comfortable", "easy to set up, works great"]
negative = ["terrible, it broke after a week", "poor quality, waste of money",
            "awful sound, very disappointed", "stopped working, returned it",
            "slow delivery and a broken box", "uncomfortable and too small",
            "bad battery, dies quickly", "useless app, keeps crashing",
            "poor fit, disappointed", "terrible customer service",
            "broke on day one, awful", "waste of time, bad quality"]
texts = positive + negative
labels = ["positive"] * len(positive) + ["negative"] * len(negative)
model = make_pipeline(TfidfVectorizer(), LogisticRegression())
model.fit(texts, labels)

new_reviews = ["the sound is great", "it broke, terrible", "fast delivery but
poor quality"]

predictions = model.predict(new_reviews)
print(model.classes_)
positive_chance = model.predict_proba(new_reviews)[0, 1]
print(predictions, round(positive_chance, 2))
```

4. The most telling words

```
import pandas as pd
from sklearn.pipeline import make_pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression

positive = ["great sound, love it", "excellent quality and fast delivery",
            "works perfectly, great price", "really happy, highly recommend",
            "comfortable and well made", "love the colour, great buy",
            "fast and easy to use", "excellent, would buy again",
            "good value, happy customer", "great battery and lovely design",
            "perfect fit, very comfortable", "easy to set up, works great"]
negative = ["terrible, it broke after a week", "poor quality, waste of money",
            "awful sound, very disappointed", "stopped working, returned it",
            "slow delivery and a broken box", "uncomfortable and too small",
            "bad battery, dies quickly", "useless app, keeps crashing",
            "poor fit, disappointed", "terrible customer service",
            "broke on day one, awful", "waste of time, bad quality"]
texts = positive + negative
labels = ["positive"] * len(positive) + ["negative"] * len(negative)
model = make_pipeline(TfidfVectorizer(), LogisticRegression())
model.fit(texts, labels)

vectorizer = model.named_steps["tfidfvectorizer"]
classifier = model.named_steps["logisticregression"]
weights = pd.Series(classifier.coef_[0],
index=vectorizer.get_feature_names_out())

top_positive = weights.idxmax()
top_negative = weights.idxmin()
print(top_positive, top_negative)
```

5. Challenge: do bigrams help?

```
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression

positive = ["great sound, love it", "excellent quality and fast delivery",
            "works perfectly, great price", "really happy, highly recommend",
            "comfortable and well made", "love the colour, great buy",
            "fast and easy to use", "excellent, would buy again",
            "good value, happy customer", "great battery and lovely design",
            "perfect fit, very comfortable", "easy to set up, works great"]
negative = ["terrible, it broke after a week", "poor quality, waste of money",
            "awful sound, very disappointed", "stopped working, returned it",
            "slow delivery and a broken box", "uncomfortable and too small",
            "bad battery, dies quickly", "useless app, keeps crashing",
            "poor fit, disappointed", "terrible customer service",
            "broke on day one, awful", "waste of time, bad quality"]
texts = positive + negative
labels = ["positive"] * len(positive) + ["negative"] * len(negative)
X_train, X_test, y_train, y_test = train_test_split(
    texts, labels, test_size=0.25, stratify=labels, random_state=42
)

uni = make_pipeline(TfidfVectorizer(), LogisticRegression())
uni.fit(X_train, y_train)
acc_uni = uni.score(X_test, y_test)

bi = make_pipeline(TfidfVectorizer(ngram_range=(1, 2)), LogisticRegression())
bi.fit(X_train, y_train)
acc_bi = bi.score(X_test, y_test)

if acc_bi > acc_uni:
    better = "bigrams"
elif acc_uni > acc_bi:
    better = "unigrams"
else:
    better = "same"
print(f"unigrams {acc_uni:.2f}, bigrams {acc_bi:.2f} -> {better}")
```